

SECDV-FINAL: Security Case Study Report

Your Name
Your Student ID

November 22, 2025

Contents

1	Introduction	2
1.1	Project Overview	2
1.2	Objectives	2
2	System Architecture	2
2.1	Technology Stack	2
2.2	Key Components	3
3	Security Analysis	3
3.1	Authentication & Authorization	3
3.2	Input Validation	3
3.3	Security Vulnerabilities & Remediations	3
4	Implementation Details	4
5	Testing	5
6	Conclusion	5
7	References	5

1 Introduction

This document presents a security analysis of the SECDV-FINAL project, an internal Event Management System (Event Tracker) built with Node.js and MongoDB. The system is designed to manage employee access, track event bookings, and generate revenue analytics.

1.1 Project Overview

The application allows an organization to manage event reservations, pencil bookings, and client details. It differentiates between three tiers of users:

- **Administrators:** Can manage all employee accounts, view audit logs, and oversee system settings.
- **Managers/Frontdesk (Role A):** Can manage event bookings, view reservations, and generate receipts.
- **Employees (Role B):** Have basic access to view schedules and update their own profile information.

1.2 Objectives

The primary objective of this case study is to secure the legacy Event Tracker application by implementing:

1. Robust Authentication and Session Management.
2. Role-Based Access Control (RBAC).
3. Comprehensive Input Validation and Sanitization.
4. Secure Audit Logging for non-repudiation.

2 System Architecture

The system follows the Model-View-Controller (MVC) architectural pattern. It handles sensitive employee data (PII) and financial transaction records.

2.1 Technology Stack

- **Backend:** Node.js with Express.js framework
- **Database:** MongoDB (using Mongoose ODM)
- **Frontend:** Server-side rendering (EJS/Handlebars) with Bootstrap
- **Authentication:** `express-session` with `bcrypt` for password hashing

2.2 Key Components

- **Admin Controller:** Manages employee lifecycle (registration, role assignment) and access logs.
- **Event Controller:** Handles the core business logic for creating, updating, and cancelling events.
- **Auth Controller:** Manages login/logout flows and session initiation.
- **Activity Logger:** A helper module that records security-critical actions (e.g., login failures, role changes) to the database.

3 Security Analysis

The following section analyzes the security controls implemented to mitigate common web vulnerabilities.

3.1 Authentication & Authorization

Authentication is handled via `auth-controller.js`. The system uses `bcrypt` to hash passwords with a salt round of 10 before storage.

- **Session Management:** Upon successful login, a session is initialized with flags `loggedIn` and `isAdmin`.
- **Access Control:** The `admin-controller.js` enforces strict checks, ensuring only authorized roles can register new employees. The constant `ALLOWED_ROLES` restricts role assignment to specific values (admin, manager, frontend).

3.2 Input Validation

Input validation is centralized in `validation.js` and `newPasswordValidator.js`.

- **Sanitization:** A `sanitizeString` helper is used to trim inputs and normalize whitespace to prevent basic injection attempts.
- **Regex Validation:** Usernames, names, and phone numbers are validated against strict regular expressions (allow-listing approach) before processing in the controller.

3.3 Security Vulnerabilities & Remediations

During the analysis of the controllers, several vulnerabilities were identified and addressed:

3.3.1 Sensitive Data Exposure

Issue: The use of `.lean()` in `getAllEmployees` previously returned full documents, including hashed passwords. **Remediation:** Queries must explicitly exclude sensitive fields using projection (e.g., `-password`).

3.3.2 Timing Attacks

Issue: The login function in `auth-controller.js` previously returned immediately if a user was not found, allowing username enumeration. **Remediation:** The logic should be updated to perform a dummy hash comparison even when the user is not found to standardize response times.

4 Implementation Details

The following code snippets demonstrate the implementation of security controls.

```
1 postRegisterEmployee: async function (req, res, next) {
2   try {
3     // 1. Input Sanitization
4     const cleanUsername = sanitizeString(username);
5
6     // 2. Validation
7     if (!isValidUsername(cleanUsername)) {
8       return res.status(400).render("...", { error: "Invalid input" });
9     }
10
11    // 3. Check for Existing User
12    const existing = await Employee.findOne({ username: cleanUsername });
13    if (existing) {
14      await activityLogger(adminUser, "Attempted to create existing account");
15      return res.status(409).render("...", { error: "Account exists" });
16    }
17
18    // 4. Secure Hashing
19    const hash = await bcrypt.hash(password, saltRounds);
20
21    // 5. Create User & Log Activity
22    const employee = await Employee.create({ ... });
23    await activityLogger(adminUsername, `Created user ${employee.username}`);
24
25    return res.redirect("/admin");
26  } catch (err) {
27    return next(err);
28  }
29 },
```

Listing 1: Secure Employee Registration with Hashing and Logging

```
1 async function logActivity(username, description, meta = {}) {
2   try {
3     await Activity.create({
4       username,
5       activityName: description,
6       meta,
7     });
8   } catch (err) {
9     console.error("Activity logging failed:", err.message);
10  }
11 }
```

5 Testing

Testing was conducted to ensure the efficacy of the security controls.

- **Unit Testing:** Validated the regex patterns in `validation.js` against malicious payloads (e.g., SQL injection strings, XSS scripts).
- **Manual Penetration Testing:** Attempted to bypass authentication using common weak passwords and brute-force methods. Confirmed that `activityLogger` successfully recorded failed login attempts.

6 Conclusion

The SECDV-FINAL project successfully hardened the Event Tracker application. By implementing industry-standard hashing, centralized validation, and audit logging, the system is significantly more resilient against common threats outlined in the OWASP Top 10. Future improvements should focus on fixing the potential Mass Assignment vulnerability in the Event Controller and implementing generic error messages to prevent information leakage.

7 References

- OWASP Top 10:2021
- Node.js Security Best Practices (Github)
- Mongoose Documentation - Schemas and Validation